

MODULE 2

Claude API & SDK Integration — Deep Dive

Complete reference for Messages API, streaming, tool use (function calling), parallel tools, extended thinking, batch API, prompt caching, vision & TypeScript SDK

Domain 1 — 22% of CCA-F Exam

Claude Certified Architect (CCA-F) | Professional Enterprise Architect | May 2026

What You Will Master in This Module

- Messages API request/response structure — every field explained with examples
- Python SDK: standard, streaming, async, error-handling, and retry patterns
- TypeScript SDK: types, async iterators, streaming with AbortController
- Tool use / function calling: single tool, multi-tool, parallel tool use
- Extended thinking & interleaved thinking (Claude 4 models — beta)
- Adaptive thinking mode and effort parameter (replacing budget_tokens)
- Prompt caching: cache_control, minimum tokens, cache hit/miss tracking
- Batch API: creating batches, polling status, processing results at scale
- Vision & multi-modal: images, PDFs, base64 encoding patterns
- Rate limits, error codes, exponential back-off, and resilience patterns
- Amazon Bedrock & Google Vertex AI SDK patterns

2.1 Messages API — Complete Request Structure

The Messages API is the single endpoint for all Claude interactions. Every request follows the same JSON structure regardless of model, modality, or feature used.

Field	Type	Required	Description / Guidance
model	string	Required	e.g. claude-sonnet-4-6 — model family + version identifier
max_tokens	integer	Required	Max output tokens. Sonnet: up to 64K. Opus: up to 128K (300K with beta).
messages	array	Required	Alternating user/assistant turn objects. Must start with 'user'.
system	string or array	Optional	System prompt — string or array of content blocks with cache_control.
temperature	float 0–1	Optional	Randomness. 0=deterministic, 1=creative default. Don't combine with top_p.
top_p	float 0–1	Optional	Nucleus sampling. Use instead of temperature, not both.
top_k	integer	Optional	Vocabulary size cap per step. Rarely needed.
stream	boolean	Optional	True = server-sent events. False = wait for complete response.
tools	array	Optional	Tool definitions (name, description, input_schema).
tool_choice	object	Optional	auto any {type:'tool',name:'X'} — controls tool selection behavior.
thinking	object	Optional	Extended thinking: {type:'enabled', budget_tokens:N} — deprecated on 4.6+.
metadata	object	Optional	{user_id:'...'} — for logging and rate limit attribution.
stop_sequences	array	Optional	Custom strings that stop generation when encountered.
betas	array	Optional	Beta features: e.g. ['interleaved-thinking-2025-05-14', 'output-300k-2026-03-24'].

Response Object — Key Fields

Every response includes these fields regardless of model or feature:

id	Message ID — store for debugging and audit trails
type	Always 'message' for standard responses
role	Always 'assistant' in the response
content	Array of content blocks: TextBlock, ToolUseBlock, ThinkingBlock, RedactedThinkingBlock
stop_reason	end_turn max_tokens stop_sequence tool_use — critical for agentic loop logic
model	Exact model string used — important for reproducibility logging
usage.input_tokens	Total input tokens consumed (includes cached tokens at full price for accounting)
usage.output_tokens	Output tokens generated
usage.cache_creation_input_tokens	Tokens written to cache this turn (billed at 125% of input price)

```
<b>usage.cache_read_input_tokens</b> # Tokens served from cache this turn (billed at ~10% of input price)
```

2.2 Python SDK — Standard, Async & Error Handling

Installation and Authentication

```
# Install pip install anthropic # Authentication — never hardcode keys in source code # Option 1: Environment variable (recommended) export ANTHROPIC_API_KEY='sk-ant-...' # Option 2: .env file + python-dotenv from dotenv import load_dotenv load_dotenv() # Option 3: Explicit (for testing only) import anthropic client = anthropic.Anthropic(api_key='sk-ant-...') # reads env var by default
```

Standard Synchronous Request — Production Pattern

```
import anthropic from anthropic import APIConnectionError, RateLimitError, APIStatusError client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY def call_claude(user_message: str, system: str = None) -> str: kwargs = { 'model': 'claude-sonnet-4-6', 'max_tokens': 2048, 'messages': [{ 'role': 'user', 'content': user_message }] } if system: kwargs['system'] = system response = client.messages.create(**kwargs) # Always log usage for cost tracking usage = response.usage print(f'Tokens: {usage.input_tokens} in / {usage.output_tokens} out') print(f'Cached: {usage.cache_read_input_tokens} read / {usage.cache_creation_input_tokens} created') print(f'Stop reason: {response.stop_reason}') return response.content[0].text
```

Async Pattern — For FastAPI / High-Concurrency Applications

```
import anthropic import asyncio async_client = anthropic.AsyncAnthropic() async def process_documents(docs: list[str]) -> list[str]: # Process multiple documents concurrently tasks = [ async_client.messages.create( model='claude-haiku-4-5-20251001', # Haiku for high volume max_tokens=512, messages=[{ 'role': 'user', 'content': f'Summarize: {doc}' }] ) for doc in docs ] responses = await asyncio.gather(*tasks, return_exceptions=True) return [ r.content[0].text if not isinstance(r, Exception) else f'ERROR: {r}' for r in responses ] # Run results = asyncio.run(process_documents(['doc1...', 'doc2...', 'doc3...']))
```

Multi-turn Conversation Management

Claude is completely stateless — you MUST send the full conversation history every request. This is one of the most common architectural errors in production deployments.

```
class ClaudeConversation: def __init__(self, system_prompt: str = None, model: str = 'claude-sonnet-4-6'): self.client = anthropic.Anthropic() self.model = model self.system = system_prompt self.history: list[dict] = [] def chat(self, user_input: str, max_tokens: int = 2048) -> str: self.history.append({ 'role': 'user', 'content': user_input }) kwargs = { 'model': self.model, 'max_tokens': max_tokens, 'messages': self.history } if self.system: kwargs['system'] = self.system response = self.client.messages.create(**kwargs) reply = response.content[0].text self.history.append({ 'role': 'assistant', 'content': reply }) return reply def reset(self): self.history = [] def token_count(self) -> int: # Estimate — use the token counting API for precision return sum(len(m['content']))//4 for m in self.history)
```

2.3 Streaming — Server-Sent Events & Real-time Output

Streaming delivers Claude's response token-by-token via server-sent events. Use it when: (1) responses are long and users should see progress, (2) you want to show typing indicators, (3) you're building interactive chat

interfaces.

Python Streaming — Context Manager Pattern

```
import anthropic
client = anthropic.Anthropic() # Context manager (recommended) – handles cleanup
# automatically with client.messages.stream( model='claude-sonnet-4-6', max_tokens=4096, system='You
# are a technical writer.', messages=[{'role': 'user', 'content': 'Write a detailed API design
# document...'}] ) as stream: # Stream text tokens as they arrive for text in stream.text_stream:
print(text, end='', flush=True) # After streaming, get the complete final message
final = stream.get_final_message()
print(f'\n\nInput: {final.usage.input_tokens} | Output: {final.usage.output_tokens}')
print(f'Stop reason: {final.stop_reason}')
```

TypeScript SDK Streaming — Async Iterator

```
import Anthropic from '@anthropic-ai/sdk';
const client = new Anthropic(); // reads ANTHROPIC_API_KEY
async function streamResponse(prompt: string): Promise {
  let fullText = '';
  const stream = await client.messages.stream({
    model: 'claude-sonnet-4-6',
    max_tokens: 4096,
    messages: [{ role: 'user', content: prompt }],
  });
  for await (const chunk of stream) {
    if (chunk.type === 'content_block_delta' && chunk.delta.type === 'text_delta') {
      process.stdout.write(chunk.delta.text);
      fullText += chunk.delta.text;
    }
  }
  const finalMsg = await stream.finalMessage();
  console.log('\nUsage:', finalMsg.usage);
  return fullText;
}
```

Event Type	What to Do With It
message_start	Opens the stream — contains model, stop_reason (null at start), usage initialized
content_block_start	Begins a new content block — index, type (text, tool_use, thinking)
content_block_delta	Token(s) within a block — delta.type: text_delta input_json_delta thinking_delta
content_block_stop	Block complete — use to finalize JSON parsing for tool inputs
message_delta	Contains final stop_reason and usage.output_tokens at stream end
message_stop	Stream completely finished — safe to call get_final_message()

2.4 Tool Use (Function Calling) — Complete Guide

Tool use enables Claude to call external functions, APIs, and services. It is the foundation of all agentic applications and is the most-tested technical skill in Domain 1.

Tool Definition Schema — Every Field

```
tools = [ { 'name': 'search_knowledge_base', # snake_case, unique 'description': ''Search the
# enterprise knowledge base for relevant documents. Use when the user asks about company policies,
# procedures, or historical data. Returns up to 5 ranked documents with relevance scores.', # Be
# very descriptive 'input_schema': { 'type': 'object', 'properties': { 'query': { 'type': 'string',
'description': 'Natural language search query' }, 'category': { 'type': 'string', 'enum': ['HR',
'Finance', 'Engineering', 'Legal', 'All'], 'description': 'Filter by document category' },
'max_results': { 'type': 'integer', 'minimum': 1, 'maximum': 10, 'default': 5 } }, 'required':
['query'] # only required fields here } } ]
```

Complete Agentic Tool Use Loop

```
import anthropic, json from typing import Any client = anthropic.Anthropic() def run_tool(name: str, arguments: dict) -> Any: '''Execute the actual tool function and return result.''' if name == 'search_knowledge_base': return search_kb(arguments['query'], arguments.get('category','All')) elif name == 'get_customer_data': return fetch_customer(arguments['customer_id']) raise ValueError(f'Unknown tool: {name}') def agent_loop(user_query: str, tools: list) -> str: messages = [{'role': 'user', 'content': user_query}] while True: response = client.messages.create(model='claude-sonnet-4-6', max_tokens=4096, tools=tools, messages=messages ) # Add assistant response to history messages.append({'role': 'assistant', 'content': response.content}) # Check if we're done if response.stop_reason == 'end_turn': return next(b.text for b in response.content if b.type == 'text') # Process all tool calls in this response if response.stop_reason == 'tool_use': tool_results = [] for block in response.content: if block.type == 'tool_use': result = run_tool(block.name, block.input) tool_results.append({ 'type': 'tool_result', 'tool_use_id': block.id, # MUST match the tool_use block id 'content': json.dumps(result) # stringify complex results }) # Return tool results as next user message messages.append({'role': 'user', 'content': tool_results})
```

Parallel Tool Use — Claude 4 Models

Claude 4 models can invoke multiple tools simultaneously when they don't depend on each other. This dramatically reduces latency for complex agent tasks.

```
# Claude may return MULTIPLE tool_use blocks in one response # Process them all before returning results: if response.stop_reason == 'tool_use': tool_calls = [b for b in response.content if b.type == 'tool_use'] print(f'Claude called {len(tool_calls)} tools in parallel') # Execute all tools concurrently import asyncio async def run_parallel(): return await asyncio.gather(*[ asyncio.to_thread(run_tool, tc.name, tc.input) for tc in tool_calls ]) results = asyncio.run(run_parallel()) # Return ALL results in a single user message tool_results = [ {'type': 'tool_result', 'tool_use_id': tc.id, 'content': json.dumps(res)} for tc, res in zip(tool_calls, results) ] messages.append({'role': 'user', 'content': tool_results})
```

Exam Tip: When Claude returns stop_reason='tool_use', you must return ALL tool results before Claude can continue. Returning only some results will cause an API error. The tool_use_id in your result must exactly match the id from Claude's tool_use block.

tool_choice Value	Behavior
auto	Claude decides whether to use tools or answer directly — default behavior
any	Claude must call at least one tool — use when you know tool use is required
{type:'tool', name:'X'}	Force a specific tool to be called — use for deterministic pipelines
none (not a parameter)	To prevent tool use entirely, don't pass the tools parameter at all

2.5 Extended Thinking & Interleaved Thinking

Extended thinking gives Claude a private scratchpad to reason through complex problems before responding. Claude 4 models (Opus 4.6+, Sonnet 4.6+) support adaptive thinking mode which is more flexible than the older manual mode.

Mode	How to Enable	Description
Manual mode (deprecated on 4.6+)	thinking={type:'enabled', budget_tokens=500}	Explicit thinking block at start of response. budget_tokens deprecated on 4.6+

Adaptive mode (recommended)	thinking={type:'auto'} + effort param	Claude dynamically decides when and how much to reason. More flexible.
Interleaved thinking (Claude 4 only)	betas=['interleaved-thinking-2025-05-14']	Thinking blocks appear between tool calls — enables reasoning about tool results
Effort parameter	effort='low' 'medium' 'high'	Replaces budget_tokens on 4.6+ — high effort = more reasoning tokens all else equal

Extended Thinking — Python Example

```
import anthropic
client = anthropic.Anthropic() # Modern approach: adaptive thinking with effort (Claude 4.6+)
response = client.messages.create(
    model='claude-opus-4-8', # Opus recommended for hard reasoning
    max_tokens=16000, # Must be >= thinking budget
    thinking={
        'type': 'auto', # Adaptive mode
        # effort='high', # Optional: high/medium/low depth
    },
    messages=[
        {'role': 'user', 'content': complex_problem}
    ]
) # Parse response - content has mixed block types
for block in response.content:
    if block.type == 'thinking':
        print('=== INTERNAL REASONING ===')
        print(block.thinking) # Show or hide from user as appropriate
    elif block.type == 'redacted_thinking':
        print('[Reasoning redacted by Anthropic safety system]')
    elif block.type == 'text':
        print('=== RESPONSE ===')
        print(block.text)
```

Interleaved Thinking with Tool Use — Claude Sonnet 4.6

```
# Sonnet 4.6: interleaved thinking via beta header
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=8000,
    betas=['interleaved-thinking-2025-05-14'],
    thinking={
        'type': 'enabled',
        'budget_tokens': 5000
    },
    tools=my_tools,
    messages=messages
) # With interleaved thinking, you'll see:
# [ThinkingBlock] -> [ToolUseBlock] -> [ThinkingBlock] -> [TextBlock]
# Claude reasons about each tool result before deciding the next action
# IMPORTANT: When returning tool results, preserve thinking blocks!
messages.append({'role': 'assistant', 'content': response.content})
# The thinking blocks in response.content must be included in subsequent turns
```

When to Use Extended Thinking

Use It For	Multi-step math/logic, complex debugging, architectural trade-off analysis, hard coding problems, ambiguous requirements
Don't Use It For	Simple lookups, summarization, format conversion, classification — thinking tokens are expensive
Cost Impact	Thinking tokens count toward output billing. High effort on Opus 4.8 can cost 10-20x a standard request.
Max Tokens Rule	max_tokens must be greater than budget_tokens. Set max_tokens=16000+ when using thinking.

2.6 Prompt Caching — Deep Dive

Prompt caching is the highest-leverage cost optimization available in the Claude API. It allows frequently repeated portions of your prompt to be cached at Anthropic's servers and served at ~10% of the input token price on cache hits.

How Caching Works

```
import anthropic
client = anthropic.Anthropic()

# Pattern 1: Cache a large static system prompt
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=1024,
    system=[
        {'type': 'text', 'text': VERY_LONG_SYSTEM_PROMPT, # Must be >= 1024 tokens
        {'type': 'ephemeral'} # Cache this block
    ]],
    messages=[{'role': 'user', 'content': user_question}]
) #

# Pattern 2: Cache a large reference document in the user message
response2 = client.messages.create(
```

```
model='claude-sonnet-4-6', max_tokens=1024, messages=[ { 'role': 'user', 'content': [ { 'type': 'text', 'text': LARGE_REFERENCE_DOCUMENT, # >= 1024 tokens 'cache_control': {'type': 'ephemeral'} }, { 'type': 'text', 'text': user_question } # Not cached ] } ) # Pattern 3: Multi-turn with cached conversation history messages = [ { 'role': 'user', 'content': [{ 'type': 'text', 'text': long_user_turn, 'cache_control': {'type': 'ephemeral'} } ] }, { 'role': 'assistant', 'content': assistant_reply }, { 'role': 'user', 'content': follow_up_question } # Only this is fresh ]
```

Cache duration	5 minutes — refreshed (for free) each time the cached prefix is used
Minimum size	1,024 tokens (Sonnet/Opus). Smaller blocks are not eligible for caching.
Cache hit price	~10% of input price: Sonnet = \$0.30/MTok, Opus = \$0.50/MTok, Haiku = \$0.10/MTok
Cache write price	125% of input price: Sonnet = \$3.75/MTok (one-time write cost, then reads are cheap)
Cache invalidation	Automatic after 5 min idle. Content change also invalidates (prefix match required).
Multiple cache points	Up to 4 cache_control breakpoints per request — cache different sections
Cache + tools	Mark tool definitions with cache_control if they rarely change
Measuring hit rate	Compare cache_read_input_tokens vs cache_creation_input_tokens in usage
Cache + streaming	Fully compatible — usage object in stream final message shows cache metrics

Caching ROI Calculator

```
# Assume: 10,000 requests/day, 5,000 token system prompt (Sonnet 4.6) # Without caching:
daily_no_cache = 10_000 * (5_000 * 3.00 / 1_000_000) # = $150/day # With caching (assume 95% cache hit rate, 5-min cache window):
cache_writes_per_day = 10_000 * 0.05 # 5% misses = 500 cache writes
cache_reads_per_day = 10_000 * 0.95 # 95% hits = 9,500 reads
daily_writes = cache_writes_per_day * (5_000 * 3.75 / 1_000_000) # $9.375
daily_reads = cache_reads_per_day * (5_000 * 0.30 / 1_000_000) # $14.25
daily_with_cache = daily_writes + daily_reads # = $23.625 # Savings: $150 - $23.63 = $126.37/day = $46,125/year
```

2.7 Batch API — High-Volume Processing

The Batch API processes large volumes of requests asynchronously at 50% off real-time pricing. It is the correct architecture for any non-real-time bulk workload.

```
import anthropic, json from pathlib import Path client = anthropic.Anthropic() # ■■■ Step 1: Prepare requests
texts_to_classify = load_documents() # list of strings
requests = [ anthropic.types.message_create_params.MessageCreateParamsNonStreaming( custom_id=f'doc-{i}', # your identifier — returned with results
params={ 'model': 'claude-haiku-4-5-20251001', # Haiku for bulk cost saving
'max_tokens': 256, 'messages': [ { 'role': 'user', 'content': f'Classify this text as POSITIVE, NEGATIVE, or NEUTRAL: {text}' } ] } ) for i, text in enumerate(texts_to_classify) ] # ■■■ Step 2: Create the batch
batch = client.messages.batches.create(requests=requests) print(f'Batch ID: {batch.id}') # Save this for polling
print(f'Status: {batch.processing_status}') # 'in_progress' # ■■■ Step 3: Poll until complete
import time while True: batch = client.messages.batches.retrieve(batch.id) if batch.processing_status == 'ended': break
print(f'Progress: {batch.request_counts.processing} processing, ' f'{batch.request_counts.succeeded} done')
time.sleep(30) # Poll every 30s — don't hammer the endpoint # ■■■ Step 4: Process results
results = {} for result in client.messages.batches.results(batch.id): if result.result.type == 'succeeded':
results[result.custom_id] = result.result.message.content[0].text else:
```

```
results[result.custom_id] = f'ERROR: {result.result.error.type}'
```

Max requests per batch	Up to 10,000 individual requests
Completion time	Within 24 hours — typically 1-4 hours for large batches
Cost savings	50% off input AND output tokens vs real-time API
Processing statuses	in_progress → ended (check request_counts for per-item success/failure)
Error handling	Each result has a type: succeeded errored canceled expired
Result access	Results available for 29 days via batches.results() streaming iterator
Best models	Haiku 4.5 + Batch API = lowest possible cost for classification/extraction
Combine with caching	Add cache_control to batch requests for additional savings on shared prefixes

2.8 Vision, PDF & Multi-modal Inputs

[illegible]

Image formats	JPEG, PNG, GIF, WebP — auto-detected from file bytes
PDF support	Up to 100 pages per document; scanned PDFs supported (OCR applied)
Max image size	5MB per image (as base64, this is ~6.7MB encoded)
Images per request	Up to 20 images in a single request
Token cost for images	~1,600 tokens per image, regardless of actual image size or resolution
URL images	Claude fetches at inference time — ensure URL is publicly accessible
Best model for vision	Sonnet 4.6 for most; Opus 4.8 for detailed analysis, chart reading
Multi-document strategy	For multiple PDFs, make separate requests or concatenate text via OCR first

2.9 Error Handling, Rate Limits & Resilience Patterns

Error	Cause	Resolution
400 Bad Request	Malformed JSON, missing required fields, invalid parameter values	Fix the request and retry; this will not succeed on retry
401 Unauthorized	Invalid or missing ANTHROPIC_API_KEY environment variable; rotate key if compromised	Check environment variable; rotate key if compromised
403 Forbidden	API key lacks permission for this feature (e.g., Batch API) and feature entitlements	Check your account tier and feature entitlements
408 Request Timeout	Request exceeded maximum allowed time	Reduce max_tokens or prompt complexity; enable streaming
429 Rate Limited	Too many requests per minute or prompt	Implement exponential back-off; use Batch API for non-real-time work
500 Internal Server Error	Transient Anthropic infrastructure error	Retry with back-off — usually resolves quickly
529 Overloaded	Anthropic is experiencing high traffic	Back off and retry; consider Batch API for tolerance

Production Resilience Pattern — Python

```
import anthropic, time, random from anthropic import RateLimitError, APIStatusError,
APIConnectionError client = anthropic.Anthropic() def robust_call(messages,
model='claude-sonnet-4-6', max_tokens=2048, max_retries=5, base_delay=1.0) -> str: for attempt in
range(max_retries): try: response = client.messages.create( model=model, max_tokens=max_tokens,
messages=messages ) return response.content[0].text except RateLimitError as e: if attempt ==
max_retries - 1: raise # Exponential back-off with jitter delay = base_delay * (2 ** attempt) +
random.uniform(0, 1) print(f'Rate limited. Retry {attempt+1}/{max_retries} in {delay:.1f}s')
time.sleep(delay) except APIStatusError as e: if e.status_code >= 500: delay = base_delay * (2 **
attempt) time.sleep(min(delay, 60)) # Cap at 60s else: raise # 4xx errors are not retryable except
APIConnectionError: time.sleep(base_delay * (attempt + 1)) # Linear for network issues
```

Tokens per minute (TPM)	Varies by tier. Default Tier 1: 40,000 TPM. Scale with usage.
Requests per minute (RPM)	Tier 1: 50 RPM. Check X-RateLimit-Remaining headers in responses.
Tokens per day (TPD)	Hard daily limits. Important for Batch API capacity planning.
Rate limit headers	X-RateLimit-Limit-Requests, X-RateLimit-Remaining-Requests, X-RateLimit-Reset-Requests
Increasing limits	Contact Anthropic sales — limits increase with consistent usage and prepayment
Batch API limits	Separate quota from real-time API — Batch does not consume real-time TPM